Decision log

Every significant choice made while building the NovaCRM onboarding agent, with the reason and the rejected alternative. Newest decisions are appended in their section. Requirements: `docs/assignment/`, design: `IMPLEMENTATION_PLAN.md`, setup: `ROCKETLANE_SETUP.md`, `CONNECTIONS.md`.

1. Principles

#	Decision	Why
P1	**Code decides, the LLM only reads and drafts.** Tier choice, template mapping, duplicate detection, escalation, migration verdicts are plain code.	Deterministic, testable, auditable. The LLM is only used where text is unstructured (email, transcript) or must be written (welcome, handoff).
P2	**No confirmed tier, no project.** Unclear / no answer → retry (max 3) → human.	The assignment's core guardrail, and the "mixed-up templates" pain.
P3	**CS owns the process.** Tier → template, duration, CSM model live in config (`TIER_RULES`), never in prompts. Templates, durations and owners live in Rocketlane.	CS can change the process without a code change or a prompt edit.
P4	**Every external action goes through one tool registry** (per-agent allowlist, pydantic validation, idempotency keys, retry, audit with masked phones).	One place to enforce and log side effects; agents can't call tools they don't own.
P5	**Anything ambiguous goes to a human escalation queue**, typed by kind (tier / input / duplicate / system / migration) with allowed resolutions per kind.	`set_tier` is only possible for tier escalations and never after a project exists.
P6	Real calls, emails and writes only when the user asks; tests and simulations use fakes.	Safety while building.

2. LLM

Decision	Why / alternatives
Claude via LangChain `ChatAnthropic`.	User asked for Claude and LangChain connectors.
Haiku 5.5 everywhere (extraction, transcript cross-check, welcome, handoff, eval judge); Sonnet/Opus only if evals show Haiku falls short.	$30 credit budget; Haiku is enough for short extraction/classification. `make eval ARGS="--model ..."` compares models.
Native structured output: `with_structured_output(schema, method="json_schema", include_raw=True)`.	The default `function_calling` forces `tool_choice`, which current Claude models reject. `include_raw` gives token usage and stop reason.
Handoff summary uses Sonnet 5.5 (`LLM_HANDOFF_MODEL`); everything else stays on Haiku.	Evals on both models: identical on extraction (26/26) and transcripts (24/24, 0 false-confirmed); welcome 3.31 vs 3.41;

Decision	Why / alternatives
	handoff 5–6/6 vs 6/6. Haiku ≈ $0.045 per full eval run vs Sonnet ≈ $0.30. Handoff is one internal document per customer (~$0.01), so quality wins there; high-volume steps stay on Haiku.
Prompt versions live in `app/llm/versions.py`, used by both production and evals; a changed prompt gets a new file (`extract_v2`, `welcome_v2`).	Evals always test what ships; audit rows record the version.
Every call logs model, prompt version, tokens, latency, cost. Refusal / unparseable → `LLMRefused` → escalation (intake) or static template (drafts).	Cost visibility; graceful degradation.
Email bodies, transcripts and task data are **untrusted data** in every prompt.	Prompt injection.
Salesforce link is found by **code**, not the LLM (LLM schema unchanged).	Validation must be exact; avoids re-running evals for a regex.

3. Workflow and orchestration

Decision	Why
LangGraph `StateGraph`: root router + intake subgraph + comms subgraph; `interrupt()` / `Command(resume)` for the call wait, retry waits and human escalation; Postgres checkpointer.	The call takes minutes; the run must survive restarts and resume on the webhook.
Nodes are re-run on resume → side effects are idempotent (keys) and interrupt-only nodes do no work.	LangGraph resume semantics.
Retry waits are durable (stored due time + scheduler `tick()`), not `sleep()` holding a lock.	Tester review 2.
Lost Retell webhooks are reconciled: after 180 s `tick()` asks Retell for the call; after 900 s a resultless call counts as failed.	Webhooks get lost.
Ambiguous write failures (timeout / 5xx after a POST) never re-POST blind: look the object up first; if the lookup also fails, stop and escalate.	Duplicate Rocketlane projects (tester review 4).

4. Intake guardrails

Decision	Why
Sender must be an **active AE in the Postgres AE directory**, checked before any LLM call or reply.	Strangers never trigger calls or replies.
Sender must be **authenticated by Gmail**: `Authentication-Results` from `mx.google.com` only (top-most), `dmarc=pass` or aligned `dkim=pass`; SPF alone never counts.	A forged `From:` would otherwise make the system phone the AE and create real projects. Verified on real inbox mail.

Decision	Why
Required fields: customer name, contact email, AE, **Salesforce opportunity link** (`*.salesforce.com` / `*.force.com` , id `006…`). Missing / invalid / conflicting → threaded reply to the AE, nothing created.	The assignment asks for clarification instead of assumptions.
Any mail from a known AE is read (AEs don't always write "Closed Won"); if it announces no signed deal (meeting request, lost or slipped deal) the run closes as `NOT_A_DEAL` with no reply or escalation.	AE mail shouldn't create noise in the human queue.
One deal per email; auto-replies ignored; duplicates detected by normalised customer name + 15-char opportunity id, locally and in Rocketlane.	Edge cases from the assignment.
Tier confirmed only if Retell's post-call analysis says enterprise/growth **and** the AE explicitly confirmed the read-back **and** an independent LLM transcript reading agrees.	A wrongly confirmed tier creates the wrong project; the transcript eval gate is 24/24 with 0 false confirmations.

5. Voice (Retell AI)

Decision	Why
Retell AI, Retell LLM agent with post-call analysis fields `plan_tier` (enum) and `tier_confirmed` (bool).	Real phone call required; structured result.
Twilio-backed US number ($2/month), **India allowed** in `allowed_outbound_country_list` .	AE (the user) is in India. Toll-free numbers can't call internationally. Card required (UPI can't pay recurring charges).
`make retell-setup` creates or updates the agent, **publishes** it and **binds the number**. Updates edit the existing LLM on a new draft version (published versions are immutable; the response engine can't be swapped).	First live call 404'd: unpublished agent, unbound number, and a hyphen in the number.
Phone settings are normalised to E.164 at load time.	Dashboard-formatted numbers broke the first call.
Pacing: voice speed 0.95, responsiveness 0.9, interruption sensitivity 0.5, short opener; Retell LLM `claude-5.5-haiku` . Prompt asks for short sentences and verbatim repeats.	Live calls: 1.0 too fast to follow, 0.8 with responsiveness 0.6 felt like a 1 s+ delay, and 0.8 interruption sensitivity chopped the opener.
Recognition: language `en-IN` , Retell `stt_mode=accurate` , boosted keywords Enterprise / Growth / NovaCRM. A garbled-but-close answer ("group plan") gets a yes/no "Did you say the Growth plan?"; a yes counts as the read-back.	Live call: with en-US/fast STT an Indian-English "Growth" came through as "group", "gurus", "Brooklyn" and two of three calls ended unclear. The guardrail held (no guess), but the AE had to take three calls. The yes/no question only ever names the plan the AE seemed to say, and still needs an explicit yes plus our transcript cross-check.
Webhook verified by HMAC signature (`v=ts,d=hex` , ±5 min).	Anyone can POST to a public URL.

6. Rocketlane

Decision	Why
Two project templates built in the UI (API can't create templates): Enterprise `5000000115814` (30 days, 16 tasks), Growth `5000000115815` (14 days, 11 tasks); 4 phases each.	Assignment: tier-specific template and timeline.
"Go-live checklist sign-off" depends on "Migration verified".	Priya's "marked done but not verified" pain.
Phase overlap kept (Configuration starts before Migration verified).	Realistic: configuration doesn't need verified data; the hard gate is go-live. User agreed.
Durations are **working days** (30-day template from 11 Nov is due 22 Dec).	Rocketlane behaviour, verified live.
Tasks are assigned to **role placeholders**; the code fills them per project (`placeholders: [{placeholderId, user:{emailId}}]`, ids resolved by name via `GET /placeholders`): Project Manager → PM, Customer success → tier CSM, Customer → contact.	Templates stay person-free; tier difference = which CSM. Verified live.
No new roles; 4 users: owner (user), `+pm`, `+csm` (dedicated CSM), sample user Rachel (pooled CSM).	Minimal setup.
Overdue escalation = two native account-wide automations (Settings → Automations): overdue by 1 day → Project Manager placeholder; by 4 days → Project owner. Not code.	Assignment says "configure an automation within Rocketlane (not a separate agent)". Account-wide covers both templates.
Project Owner vs Project Manager: Owner = the accountable person on the project (built-in field); PM = the person running the plan (placeholder).	Nudge first, escalate second.

7. Slack

Decision	Why
Dedicated **NovaCRM** workspace; app created from a manifest with bot scopes `channels:manage`, `channels:read`, `chat:write`, `conversations.connect:write`, `users:read`, `users:read.email`; **token rotation off**.	Admin rights, clean demo; rotation would expire the token every 12 h.
Channel name, topic and welcome are tier-personalised; the welcome is LLM-drafted, fact-checked in code (name, duration, CSM model; no prices or dates), one regeneration, then a static template.	Assignment requirement + no hallucinated commitments.
Slack Connect invite to the customer is attempted; `not_paid` etc. degrade gracefully (internal channel + audit note).	Free workspaces can't use Connect.
The Salesforce link and anything internal never go into the customer-shared channel.	The channel is customer-facing.

8. Gmail

Decision	Why
OAuth desktop client, consent screen in **Testing** mode, scope `gmail.modify`, refresh token in `.env`; access tokens are minted in memory by the client library.	Simplest working auth. Testing-mode refresh tokens expire after 7 days → re-run `make gmail-auth`.
CS inbox = the account's real address (not a `+alias`).	Google identifies the account, not the alias; it's also the From of our replies.
Push: Gmail `users.watch` → Pub/Sub topic `gmail-deals` → push subscription → `POST /webhooks/gmail?token=…`. **20 s polling stays on** as a fallback; runs dedupe by message id.	Instant trigger without losing mail if a push is dropped. Watch expires after 7 days → `POST /admin/gmail/watch` (production: daily scheduler).
`history.list` cursor in Postgres; 404 skips a message; `history_too_old` → 2-day full sync.	Sync survives deleted messages and expired cursors.

9. Data migration verification

Decision	Why
Five code checks between two real Postgres tables (`mig_source.contacts` vs `mig_dest.contacts`): record count, missing PKs, duplicates, blank required fields, and columns (every source column must exist in the destination; extra destination columns are fine). The demo tables differ like a real CRM migration: source 60 rows × 7 columns (old CRM, with blank optional phones/titles), destination 57 rows × 7 different columns (`phone` dropped, `lifecycle_stage` added), 4 rows never copied, 1 duplicated, 2 blank emails, 1 blank name.	Priya's "marked done but not verified".
Evidence goes into the "Migration verified" task's **private note** (no comment API); the agent **never completes the task** — a person does. Failure → `migration` escalation to the CSM; dismissable with a recorded reason.	Human stays accountable; evidence is where the PM works. Verified live: status/dates untouched.

10. Deployment and operations

Decision	Why / alternatives
AWS, personal account (profile `jk-personal`), one EC2 `t3.small` (free-tier eligible) with Docker Compose: API + Postgres + Caddy.	User's free credits. ECS Fargate + RDS + ALB ≈ $40+/month; ngrok/cloudflared would break when the laptop sleeps.
HTTPS via Caddy + Let's Encrypt on `<ip>.sslip.io`.	Retell and Pub/Sub need HTTPS; no domain purchase.
Everything except `/webhooks/*`, `/healthz` and the login page is behind an in-app login (`app/auth.py`: signed HttpOnly SameSite=Strict session cookie, 12 h; same-origin check on POSTs; 5 failed logins per IP per 5 min; fails closed without `ADMIN_PASSWORD`). Replaced Caddy basic auth (browser popup). Webhooks authenticate themselves (Retell HMAC, Gmail push token).	`/runs` and `/resolve` expose customer data and can trigger actions.

Decision	Why / alternatives
Secrets: `.env` is copied to the host over SSH (`chmod 600`), never committed; Postgres and admin passwords are generated on the host.	No secrets in git or images.
Logs: one JSON object per line, phones masked; request log in Caddy with the push token and auth headers redacted; uvicorn access log off.	The push token travels in the query string.
Single uvicorn worker.	The Gmail poller and scheduler run in-process; more workers would duplicate them.
Production path (not built): managed Postgres, Secret Manager / SSM, a real domain, scheduled watch renewal, multiple workers with the poller/scheduler moved to a separate process, OIDC-authenticated Pub/Sub push.	Said in the video.

11. Testing and quality

Decision	Why
TDD; `pytest` with fakes only, `pytest-socket` blocks network; Postgres contract tests run the same suite on MemoryStore and PgStore.	Fast, deterministic, real-DB parity.
`make simulate SCENARIO=all` : end-to-end mock runs (happy, no answer, ambiguous, API down, duplicate, …).	Assignment's edge cases, runnable in seconds.
`make eval` : real-Claude datasets (extraction, transcripts, welcome); transcript accuracy gate 1.0, a false "confirmed" is critical.	Quality of the LLM parts is measured, not assumed.
A dedicated **tester** subagent reviews every feature before it's called done; its defects are fixed with regression tests.	Independent review.
No `Co-Authored-By` or other trailers in commits.	User rule.

11b. Agent boundaries

Agent	Owns	Cannot
1 Intake & Routing	understanding the deal, confirming the tier by phone, creating the project	post anything to Slack
2 Communication	every customer-facing message: channel, topic, invites, welcome, phase progress updates (each checked for internal wording and prices before posting)	decide tier, create projects, change tasks
3 Project Tracking	watching Rocketlane, migration verification, reopen, handoff	reach a customer channel (it asks Agent 2; its only Slack tool, `slack_post_internal` , refuses any channel that belongs to a customer run)

Why a separate Communication agent when its first job is "create a channel": agents here are ownership and permission boundaries, not separate LLMs. The agent reading untrusted email can't message customers, a Slack outage doesn't roll

back a project, and one place enforces what customers may be told.

12. Tracking agent (Agent 3) and dashboard

Decision	Why
Agent 3 — Project Tracking: a deterministic watcher (scheduler every `TRACKING_INTERVAL_SECONDS`, default 600, plus `POST /runs/{id}/sync`) that diffs Rocketlane task snapshots and acts.	Priya's process continues after setup: phases, verification, handoff. Polling is simple and reliable; Rocketlane webhooks would need public-endpoint setup per account.
Rules: phase complete → one customer-safe Slack line; import task complete → auto migration check; "Migration verified" ticked without a passing check → reopen + escalate; all Go-Live done and migration resolved → LLM handoff summary (fact-checked, static fallback) in the "Handoff to support" private note, run → `HANDED_OFF`.	Code decides; the LLM only writes the summary. The handoff is internal, never in the shared channel.
What people do in the Rocketlane UI beyond ticking tasks (verified live): a **phase marked Completed while its tasks are open** (Rocketlane allows it and leaves the tasks open) and a **task or phase set to Blocked** → a `tracking` item in *Needs a human* plus one post in the internal channel; it clears itself once fixed in Rocketlane. The phase is announced to the customer only when its tasks are done.	Same principle as "Migration verified": a status without the work behind it is not progress, so the customer is never told something that isn't true. Fixing it where the work lives (Rocketlane) closes it; there is no dashboard button to wave it away.
Not watched: **comments / task chat** and the **At risk** flag.	Rocketlane's public API has no comment endpoint, and `atRisk` is write-only (rejected in `includeFields`, absent from every GET; checked live). Blocked status is the readable way to say "stuck".
The handoff summary is also posted to an **internal** Slack channel (`SLACK_HANDOFF_CHANNEL`, `#cs-handoffs`), best effort and once; refused if it equals the customer channel.	Support lives in Slack; the customer-shared channel never gets internal notes.
Overdue escalation is **not** in Agent 3.	The assignment requires a native Rocketlane automation; duplicating it would double-notify.
Tasks are read with `includeFields=phase` (phase is null otherwise); statuses To do=1, In progress=2, Completed=3.	Verified against the live API.
Dashboard at `/ui`: plain HTML/CSS/JS served by FastAPI (no build step, no CDN), light theme, larger type, 7-step pipeline per run, paginated lists (5/10/25), side-panel escalation actions, tabs, polling every 3 s. Behind the same in-app login (sidebar layout, user menu with sign out). Designed with the taste-skill guidance and beautifului.dev patterns.	Ships with the API on the same host; shown live in the demo while the call happens. Data is rendered with `textContent` only (emails/transcripts are untrusted).

13. Idempotency (after the audit)

Decision	Why
Idempotency keys are **reserved as in-flight before** every side effect. An in-flight key found later is reconciled by a lookup (Rocketlane by externalReferenceId, Slack channel by name, Slack post by history, Gmail by Message-ID) or, if no lookup exists, escalated as "outcome unknown".	The audit showed "record after success" repeats calls/projects/messages after a crash or timeout.
Retell create-call sends an idempotency key per (run, attempt).	A timeout must not phone the AE twice.
Opportunity duplicate check runs **before** the call as well as before the create.	Don't phone an AE about a deal that's already onboarded.
Single uvicorn worker is a hard requirement until the in-process lock is replaced by a Postgres advisory lock.	All concurrency guarantees assume one process.